

Developing NailTech: A Data-Driven Nail Salon Booking and Analytics System

Justin Tran

CSS 496 Capstone Project

University of Washington Bothell

Spring 2026

Table of Contents

1. Project Overview	3
1.1 Project Description and Purpose	3
1.2 Problem Statement	3
1.3 Solution Approach	4
1.4 Primary Goals and Objectives	5
1.5 Target Audience and Users	6
1.6 Current Project Status	6
2. Technical Implementation	6
2.1 Key Technical Requirements	6
2.2 System Architecture and Design	7
2.3 Technologies and Tools	9
2.4 Current Technical Components Implemented.....	10
2.5 Testing and Validation	14
2.6 Remaining Technical Work	16
3. Development Process Reflection	16
3.1 Project Management Approach	16
3.2 Project Evolution	17
3.3 Key Decisions and Rationale	17
3.4 Feedback and Iteration	18
3.5 Significant Changes During Development	20
4. Challenge Analysis and Problem-Solving.....	20
4.1 Major Challenges.....	20
4.2 Problem-Solving Approach.....	22
4.3 Lessons Learned from Challenges	22
4.4 Alternative Solutions Considered.....	22
4.5 Future Improvements to Workflow	22
5. Learning and Professional Growth.....	22
5.1 Technical Skills Developed.....	22
5.2 Personal and Professional Growth.....	23
5.3 Connection to Coursework.....	23

5.4 Career Impact.....	23
5.5 Biggest Takeaways.....	24
6. <i>Key Insights and Future Directions</i>	24
6.1 Most Important Takeaways.....	24
6.2 Most Meaningful Part of the Project	24
6.3 Future Improvements	25
6.4 Real-World Applications.....	25
6.5 Advice for Future Students.....	25
6.6 What I Would Do Differently	25
7. <i>References</i>	25
8. <i>Appendices</i>	26

1. Project Overview

1.1 Project Description and Purpose

This capstone project involves developing a full-stack web application for a nail salon that allows customers to browse services and request appointments online. The system will also store booking data and analyze it to identify patterns such as peak hours, busiest days, and most requested services, and translate these patterns into actionable recommendations to support business operations.

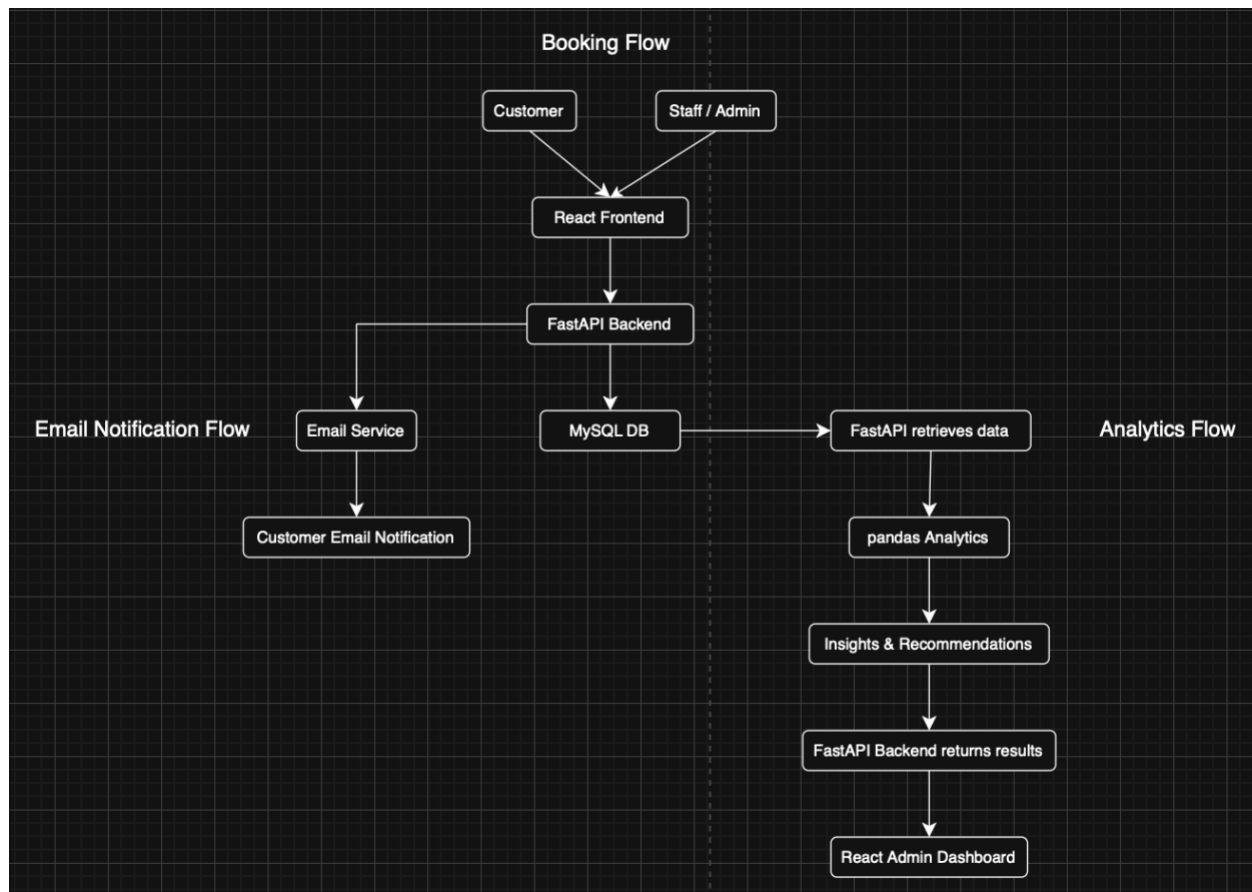
The primary beneficiary of this system is the salon owner, who will gain insights into business operations and make more informed decisions about scheduling, promotions, and service offerings. Customers will benefit from a more convenient booking experience, while nail technicians will gain better visibility into busy and slow periods. The system will include separate customer-facing and staff/admin interfaces, allowing customers to submit booking requests and staff to manage bookings when needed.

1.2 Problem Statement

The problem with how booking worked in the salon was that they were tracked manually on a paper planner. At times, appointment information such as customer names, phone numbers, dates, times, and requested services could be recorded incorrectly or omitted entirely. Not only

that but when the salon is busy, phone calls could be missed losing out on an appointment. Overall, the appointment management process could be improved through a centralized digital system. And since no booking data was tracked in an automated way like a database, it was difficult to track business trends; that meant that there was no centralized system.

1.3 Solution Approach



The system follows a full-stack architecture consisting of a React frontend, a FastAPI backend, and a MySQL database, along with components for analytics and email notifications.

Customers and staff interact with the React frontend to submit and manage appointment requests, which are handled by the FastAPI backend and stored in the MySQL database. The backend retrieves booking data and processes it using pandas to identify patterns such as peak hours, busiest days, and service demand. These insights are translated into actionable recommendations and returned to the frontend dashboard for visualization.

The system also includes an email notification component, where the backend communicates with an external email service to send appointment status updates (e.g., accepted or declined) to customers.

The completed system was deployed using Vercel for the frontend, Railway for the FastAPI backend, and Railway MySQL for database hosting. Cloud deployment allows users to access the application remotely while maintaining communication between the frontend, backend, database, analytics components, and email notification system.

In this workflow, customers submit appointment requests through the frontend, which are processed by the backend and stored in the database. Staff then review each request and either approve, decline, or contact the customer directly if adjustments are needed. Customers will receive email notifications when their appointment status is updated.

1.4 Primary Goals and Objectives

- Design and implement a full-stack web application with an appointment request system
- Build and manage a relational database to store services and booking data
- Develop an admin/staff dashboard for viewing, reviewing, and managing appointment requests
- Visualize booking and service data through charts and summaries to support analysis and decision-making
- Analyze booking data to identify patterns such as peak hours, busiest days, and service demand to support business insights
- Generate data-driven, actionable recommendations that support business decisions such as staffing, promotions, and service prioritization
- Develop separate customer and staff/admin interfaces so customers can request appointments, and staff can manage bookings and view analytics.
- Implement email notifications for appointment status updates
- Deploy the application using cloud hosting platforms to support real-world accessibility

1.5 Target Audience and Users

- Salon owners/staff
- Customers
- Nail technicians

The system is primarily designed for salon owners and staff who need a more organized way to manage appointments and analyze booking trends. Customers benefit from a more convenient booking process, while staff gain better visibility into scheduling and service demand.

1.6 Current Project Status

The project has successfully achieved its primary objectives and is fully functional.

Completed Components:

- Customer appointment request workflow
- Admin appointment management dashboard
- Analytics dashboard with visualizations
- Recommendation generation system
- Search and filtering functionality
- MySQL database integration
- Email notification system
- Customer and admin interface separation
- Cloud deployment using Vercel and Railway

The completed system demonstrates the full booking lifecycle from appointment submission through appointment management, analytics generation, and customer notifications.

Overall, the project successfully achieved the major goals established at the beginning of development and demonstrates a complete full-stack booking and analytics solution for a real-world nail salon environment.

2. Technical Implementation

2.1 Key Technical Requirements

- Booking request system
- Admin management workflow
- Analytics dashboard

- Database integration
- Responsive frontend UI
- Email notification integration
- Cloud deployment

2.2 System Architecture and Design

The system is organized into frontend, backend, and database components.

Frontend

- React + Vite
- Chart.js
- Admin/customer pages

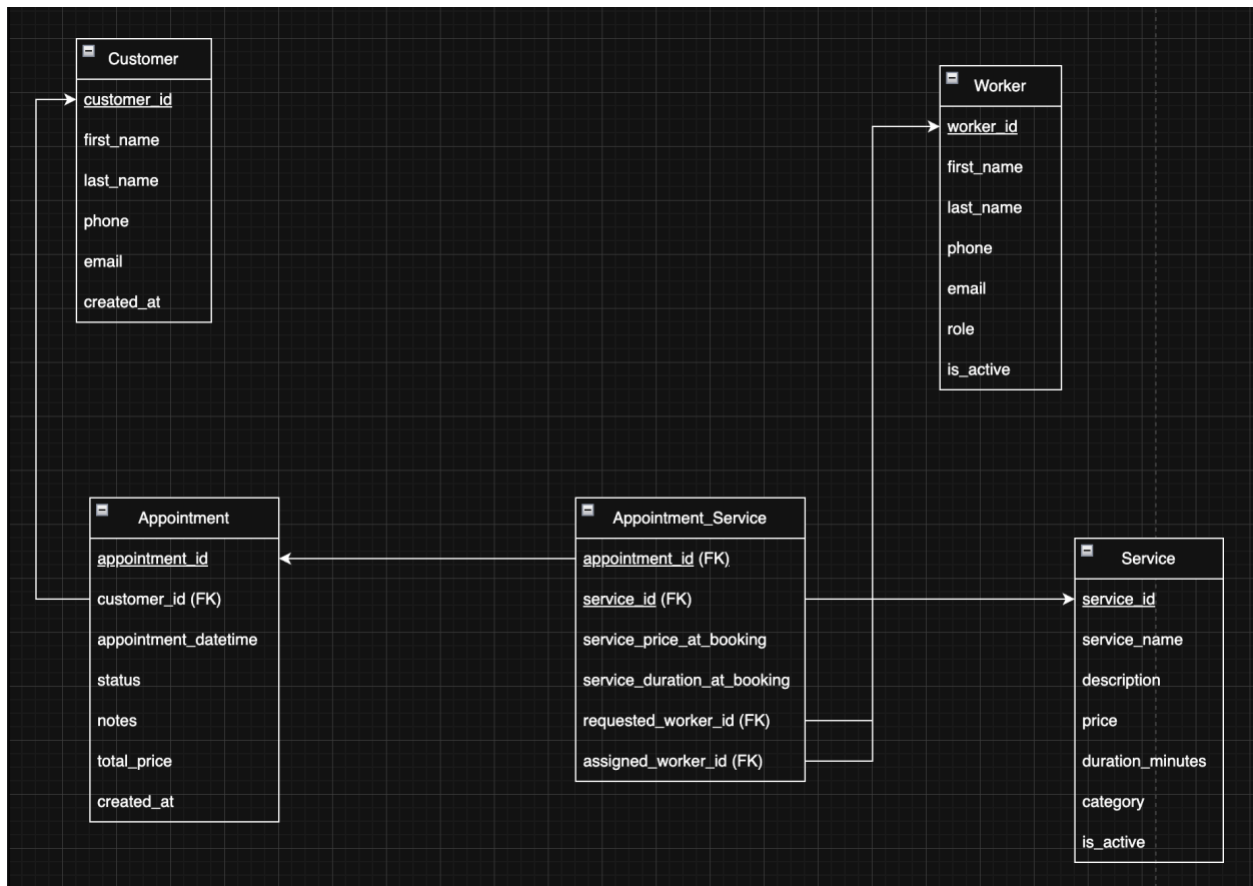
Backend

- FastAPI
- API endpoints
- Booking logic
- Analytics endpoints

Database

- MySQL
- Tables:
 - customers
 - appointments
 - workers
 - services
 - appointment_services

Database Design



The database was designed around the appointment booking workflow and includes five main tables: customers, workers, services, appointments, and appointment_services. The customers table stores customer contact information, while the workers table stores salon staff information such as name, role, and active status. The services table stores the salon service menu, including service name, description, price, duration, category, and active status.

The appointments table stores the main appointment record, including the customer, appointment date and time, status, notes, total price, and creation time. Each appointment belongs to one customer, while one customer can have many appointments. This structure supports appointment history, repeat customer tracking, and analytics based on customer booking behavior.

The appointment_services table is a table made between appointments and services due to the many-to-many relationship that they have. This table was necessary because one appointment can include multiple services, and each service can appear in many different appointments. It

also stores additional booking details such as requested worker, assigned worker, service price at booking, and service duration at booking. Storing price and duration at the time of booking preserves historical appointment information even if the service menu changes later.

This database design reduced unnecessary duplication while supporting the booking workflow, admin dashboard, analytics dashboard, and future system expansion. Figure B1 in Appendix B shows the complete database schema and relationships used by the application.

The frontend communicates with the backend through REST API endpoints which are developed in FastAPI. Customer actions like submitting appointment requests trigger API calls that create and return records from the MySQL database. Administrative actions like reviewing appointments and viewing analytics also rely on the backend endpoints to retrieve and process data.

The analytics dashboard also shares this architecture by pulling appointment data from the database and then using Python and pandas to process it. Frontend returns this data through charts via Chart.js.

The application was designed with separation in mind. For example, the frontend is only responsible for user interaction and presentation. While the backend handles all the business logic and data processing. And the database serves as the storage. This architecture made the system easier to develop, test, and maintain while allowing each component to be modified without having an impact on other parts of the application.

2.3 Technologies and Tools

- React: This application was built on react because it is easy to fetch and display live data from the FastAPI backend. React also supports reusable UI components. It allows for dynamic filtering, forms, and interactive CRUD functionality.

- FastAPI: FastAPI also integrates seamlessly with Python, which I am comfortable with. FastAPI was used as the backend because it is beginner-friendly and easy to use for routing API requests and database operations. FastAPI also automatically generates a /docs page for API testing and integrates seamlessly with Python.
- MySQL: MySQL was selected because it supports relational database features such as foreign keys, constraints, indexing, and structured queries.
- Chart.js: Used to create interactive visualizations for appointment trends, service demand, peak booking periods, and other analytics displayed in the admin dashboard.
- SQLAlchemy: Used as the ORM between FastAPI and MySQL. SQLAlchemy simplified database operations, allowed database models to be defined in Python, and helped manage relationships between customers, appointments, workers, and services.
- Python: Used throughout backend development for API creation, business logic, database interactions, analytics calculations, recommendation generation, and email notification functionality.
- GitHub: Version control
- VS Code: I am comfortable with VS Code and it supports:
 - Python & FastAPI development
 - React development
 - MySQL extensions
 - Git/GitHub integration
 - Virtual environment support
- Railway
- Vercel

2.4 Current Technical Components Implemented

- CRUD booking workflow
- Admin dashboard
- Analytics dashboard
- Search/filter functionality
- Charts and recommendations
- Email notification integration
- Customer/admin interface separation

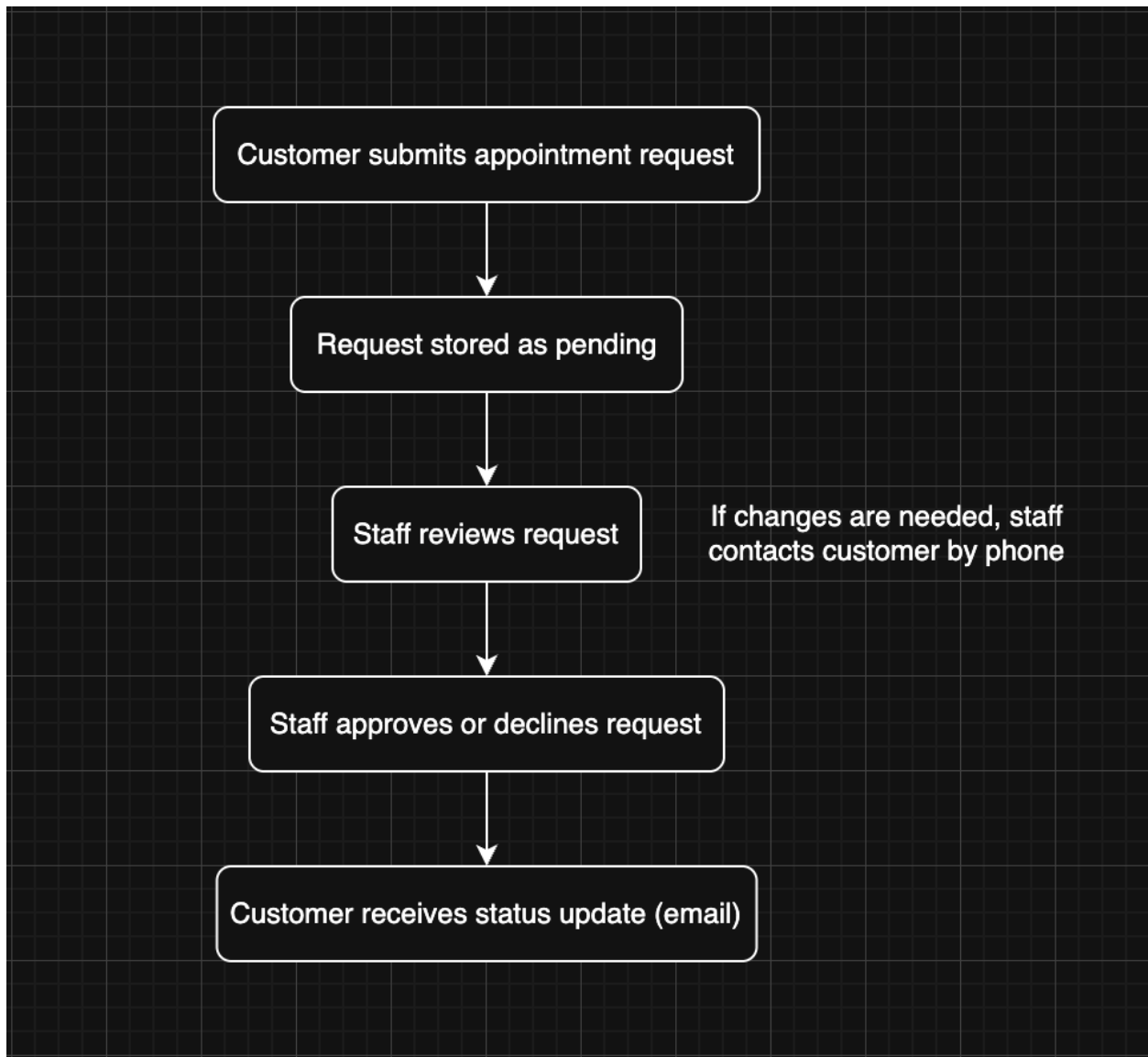
- Recommendation engine
- Analytics filtering
- Cloud deployment (Vercel + Railway)

The completed system supports the full appointment lifecycle. Customers can browse services and submit appointment requests through the frontend. Staff members can review, approve, decline, complete, or cancel appointments through the admin dashboard. Analytics features provide insights into booking activity through summary cards, visualizations, and recommendations. Email notifications improve communication between the salon and customers. Deployment to Vercel and Railway allows the application to be accessed remotely through a cloud-hosted environment.

Customer Interface

The customer interface was designed to be simple and easy to navigate. Customers can browse services, review pricing information, and submit appointment requests through an online booking form. The interface was kept minimal as possible to reduce confusion and make the booking process as straightforward as possible.

Booking Workflow



The booking workflow was designed around a request-and-review process. Customers browse available services and submit appointment requests through the booking form. When a request is submitted, the system checks for an existing customer record using email or phone number. If a matching customer is found, the customer information is updated rather than creating another record.

New appointment requests are stored with a pending status. Staff members can then review each request through the admin dashboard and choose to approve, decline, cancel, or complete the

assignment. If staff sees that the pending appointment request doesn't work, they then can manually look at the customer information and contact them to further assist them in booking an appointment that works. This workflow allows staff to maintain full control over scheduling while still providing customers with a convenient online booking system.

Screenshots of the booking workflow are included in Appendix D.

Admin Dashboard

The admin dashboard serves as the central management interface for salon staff. Here is where the staff member looks at all the appointments and are able to review them and to have control of actions like approve, decline, cancel, and complete. This page shows all appointment information like id, customer, phone, email, services/requested worker, status, and notes. The list that is sorted by pending and earliest dates first. There is also a filtering option that makes it easier to filter by name, phone number, email, statuses (pending, approved, declined, cancelled, completed, and/or by date.

From this page, staff members can navigate to the add booking page and analytics page.

Add Booking Page (Staff)

This page is exactly the same as the booking page from the customer side, however instead of the appointment request going in as pending, it will go in as approved.

Analytics Dashboard Design

The analytics dashboard was designed to turn the appointment data into useful business insights. There are summary cards that show information like total appointments, completed appointments, completed revenue, repeat customers. Several visualizations were developed to help identify and visualizes trends including busiest days, peak booking hours, service category demands, top services, and top requested workers.

To make the dashboard more useful, a filtering feature for the whole page was implemented allowing users to analyze recent activities from all time to last 7 days, last 30 days, and last 90 days. And at the end is where the recommendation feature is. There are a max of 5 recommendations at a time that translate the raw data into actional business suggestions that relate to staffing, scheduling, and service demand.

The analytics dashboard relied on backend analytics endpoints that aggregated appointment data and returned the results to the frontend. These endpoints calculated metrics like busiest days, peak booking hours, most requested services, service category demand, and top requested workers. This approach of separating the analytics from the frontend visualizations makes it easier to expand the analytics functionality in the future if need be.

Example dashboard visualizations can be found in Appendix F.

2.5 Testing and Validation

Database Testing

Database testing was done to ensure that the tables, relationships, and constraints functioned correctly. Making sure that customers, workers, services, appointment, and appointment_services tables and primary keys, foreign keys were implemented correctly. A few CRUD operations were used to test that the database records would be inserted, retrieved, updated, and deleted correctly.

Additional SQL join queries were used to test and make sure that the relationship between the tables functioned properly. This was important because one appointment could have multiple services in the appointment_services table. Testing helped confirmed crucial information like appointment details, service assignments, pricing, and customer information were stored and retrieved correctly.

Realistic Test Data Generation

Testing involved generating realistic appointment data to simulate actual salon activity., Since the project was developed within a single academic quarter, there was not enough time to compile real booking data or to spend time to manually go through the physical paper calendar booking book to be able to fully demonstrate that the analytics features work. Although it would have been a nice touch, it wasn't time efficient. To address this issue, realistic test data was generated to represent the customers, appointments, services, booking trends, and worker request. The dataset was large enough to allow the analytics dashboard, charts, filtering functionality, and recommendation features to be tested realistically as possible.

Backend Testing

Testing was performed throughout development using multiple approaches. FastAPI's automatically generated API documentation was used to verify backend routes, CRUD operations, analytics calculations, and database interactions. For the frontend testing, that involved manually validating booking request, appointment management workflows, filtering functionality, and analytics visualizations. Database testing was used to ensure that data integrity across customers, appointments, services, and workers. Additional testing was conducted after deployment was successful to verify that the the communication between Vercel frontend, Railway backend, Railway MySQL database, and email notification system had worked.

Frontend Testing

Specific testing scenarios included submitting appointment request, verifying database inserts, and approving and declining the appointments though the admin dashboard, validating analytics calculations, testing date filtering functionality, and verifying email notifications. Testing was performed throughout development and not just at the end of the project so that issues can appear and be resolved right away instead of having them build up.

Deployment Testing

The application was deployed using Vercel for the frontend and Railway for the backend and MySQL database. Testing performed to make sure that the deployed frontend could communicate with the backend, the backend could connect to the database, and all main features functioned after deployment.

Most core features functioned correctly after deployment including the booking request workflow, admin appointment management, analytics data retrieval, and database integration. Email notification functioned via testing through the deployed FastAPI backend documentation page, where the backend email endpoint was able to successfully trigger emails to be sent.

However, the frontend-to-email workflow did not work in the deployed environment. There is likely some issue with either the environment configuration or cloud hosting limitations and not with email utility.

Although this issue was not fully resolved, the core functionality of the application works in the deployed environment.

2.6 Future Technical Improvements

Potential future enhancements include:

- User authentication and role-based access control
- SMS appointment reminders
- Worker scheduling and availability management
- Customer appointment history tracking
- Additional analytics metrics and forecasting features
- Performance optimization through aggregated API endpoints
- Further separation of customer and staff interfaces
- Automated appointment approval workflows for eligible requests

3. Development Process Reflection

3.1 Project Management Approach

I mainly approached the project through incremental development by focusing on a feature at a time. Weekly reports, instructor meetings, stakeholder feedback, expert feedback, and testing helped guide the direction and organization of my project.

3.2 Project Evolution

(Started as booking system → evolved into analytics-focused full-stack system)

When I first thought about what project I wanted to make for my capstone I was working at the nail salon in the summer and noticed that the shop didn't have a website, and I could make a website. But that didn't fully connect with what I was studying and my second discipline so I thought about what other real-world problem there was that I could solve. Then I thought about how booking at the salon was done manually on paper so I wanted to develop a system that could automate that process a little bit. Which finally landed me at the idea that would require a and with that comes with tracking data and I can use that data to be able to provide business insights. I then finally had the idea I wanted to create, and it connects to my data science / analytics minor.

As development continued, the booking workflow became the foundation for the rest of the project. Once the appointment process was designed, it became easier to determine what information needed to be store within the database. From there, I was able to identify which data points would be the most useful for analytics and business insights. The stakeholder feedback also helped guide this process by highlighting which metrics would be the most valuable from a salon management perspective. This allowed the analytics dashboard to evolve naturally from the booking system rather than being treated as a separate feature.

3.3 Key Decisions and Rationale

Several technology decisions were made based on prior experience and project requirements.

React was selected because I wanted additional experience building dynamic frontend applications. React supports reusable components, state management, and interactive user interfaces, making it well suited for a booking platform.

FastAPI was selected as the backend framework because it is lightweight, easy to develop with, and integrates naturally with Python. FastAPI also provides automatic API documentation through the /docs interface, which simplified testing throughout development.

MySQL was selected because of my previous experience using relational databases. Since the application relies heavily on relationships between customers, appointments, workers, and services, MySQL provided a reliable and familiar solution.

Overall, I selected a technology stack that I had previously used in a smaller full-stack project. This allowed me to spend more time expanding my skills and implementing new features rather than learning an entirely new set of technologies.

3.4 Feedback and Iteration

Instructor Feedback

During the instructor meeting, I presented the current progress of the project, including the services page, booking workflow, admin dashboard, and analytics page. I explained how customers can submit booking requests and how staff can manage appointments through the admin side of the system. I also discussed the analytics and recommendation features that connect the project to my Data Science minor.

The instructor said the project and intelligent/data-driven aspects were moving in a good direction. Feedback from the meeting reinforced the importance of continuing to improve the analytics dashboard, adding more visualizations, and better separating the customer and admin sides of the frontend. The meeting also helped confirm that the project scope and overall direction were appropriate for the capstone.

Stakeholder Feedback

The stakeholder explained that the salon currently handles most appointments through phone calls and paper scheduling. One of the main issues mentioned was that phone calls can be missed during busy periods, which may result in lost booking opportunities. The stakeholder also shared that weekends are usually the busiest times and that it is important for staff to quickly view important appointment information such as customer name, phone number, services, worker, date, and time.

After reviewing the booking form and admin dashboard, the stakeholder said the system was easy to use and did not suggest any major changes. This feedback helped reinforce the

importance of keeping the system simple, organized, and practical for real salon use while continuing to improve the booking workflow and dashboard organization.

Additional Stakeholder Feedback

During a later stakeholder meeting, I presented the analytics dashboard and discussed how the system could support business decision-making. The stakeholder said the analytics were useful for visualizing salon activity but would become even more valuable as more booking data is collected over time. Additional suggestions included adding employee-specific analytics, filtering options for date ranges and workers, and tracking returning customers.

This feedback helped me better understand which analytics would be most useful in a real salon environment. Based on the discussion, I implemented analytics filtering features and identified additional business-focused analytics, such as repeat customer tracking and staff performance metrics, as future improvements.

Expert Feedback

The expert mainly gave feedback about improving the separation between the customer side and the staff/admin side of the system. They explained that customers and staff should have different experiences and suggested making the navigation and workflow clearer. They also recommended adding a better entry point for staff instead of only using the /admin route.

This feedback helped me think about how the system would work in a real salon setting. Based on the meeting, I improved the homepage, frontend organization, and separation between customer and admin features while still keeping the project manageable for the capstone scope.

Additional Expert Feedback

During a follow-up meeting, I demonstrated the analytics dashboard and discussed how the analytics could be expanded further. The expert suggested adding more advanced analytics related to employee booking performance, additional filtering options, and a larger testing dataset to better demonstrate the usefulness of the dashboard. They also recommended continuing to improve the overall admin dashboard workflow and realism of the data being analyzed.

This feedback reinforced the importance of designing analytics that support real business decisions rather than simply displaying information. As a result, I continued improving the analytics dashboard, implemented filtering capabilities, and considered additional employee-focused analytics as future enhancements.

Overall, feedback from the instructor, stakeholder, and expert played an important role throughout development. Their suggestions influenced improvements to the analytics dashboard, customer and admin interface separation, frontend organization, filtering functionality, and future planning for advanced business analytics features. The feedback process helped ensure that the final system remained practical, user-friendly, and aligned with real-world salon needs.

3.5 Significant Changes During Development

The overall project idea remained mostly consistent throughout development, since I already knew I wanted to create a booking and analytics system for my mom's nail salon that also incorporated my Data Science & Analytics minor. Most of the changes during development involved refining the booking workflow, frontend organization, and admin/customer experience. For example, earlier ideas included allowing customers to check the status of their booking requests directly through the website. However, after feedback and further planning, I decided to simplify the workflow by focusing more on staff-managed booking updates and email notifications. Feedback from instructor, stakeholder, and expert meetings also influenced improvements to the analytics dashboard, homepage planning, and separation between customer and admin views.

4. Challenge Analysis and Problem-Solving

4.1 Major Challenges

- Database design
- Connecting frontend/backend/database
- Analytics implementation
- UI organization
- Managing project scope
- Cloud deployment and environment configuration

- Cross-platform integration

Database design was challenging because the system needed to support customers, appointments, workers, services, analytics, and future scalability while also minimizing unnecessary duplication of data. Determining the relationship between appointments and services required careful planning because a single appointment could contain multiple services and worker requests.

Another challenge involved integrating the frontend, backend, database, analytics components, and deployment environment. Since each component depends on each other, an issue on one layer would affect other parts of the system. This made it very difficult to troubleshoot issues and required a lot of testing and debugging throughout development.

Deployment and Cloud Configuration

Deployment was a main challenging factor in this project since multiple services were used to deploy this application. This required more configurations to ensure that all components work after deployment.

One issue was the email notification feature. Emails were able to be sent through the deployed FastAPI backend but did not work through the deployed frontend form. The individual components worked through testing, but when it came to the complete workflow in the cloud environment, it didn't.

Another challenge was the limitations with the free-tier hosting services. Some pages loaded significantly more slowly compared to running the application locally. This was due to the fact of how many API requests were needed to retrieve data. While these issues did not prevent the application from functioning, it did create challenges while deploying and testing.

4.2 Problem-Solving Approach

To solve problems during development, I used testing, debugging, research, and iteration. I tested my backend API routes often through the FastAPI / docs and adjusted frontend and database logic based on issues that appeared during development.

4.3 Lessons Learned from Challenges

One important lesson learned during development was the importance of planning system architecture and frontend organization early. As the application grew, managing interactions between the frontend, backend, database, analytics components, and deployment environment became increasingly complex. Proper planning can reduce future redesign work and improve development efficiency.

Another important lesson was that an application may function correctly when ran locally, there could be many issues when it comes to deploying. Cloud hosting can introduce issues like API communication, environment variables, and performance limitations. It is important to not rely only on local testing but to do deployment testing and to test complete user workflows.

4.4 Alternative Solutions Considered

One alternative idea considered during development was allowing customers to directly check their booking request status through the website. However, I later decided to simplify the workflow by focusing more on staff-managed updates and email notifications.

4.5 Future Improvements to Workflow

If I continued this project, I would spend more time planning the frontend structure and overall workflow earlier before implementing additional features. I would also begin deployment and UI testing earlier in development.

5. Learning and Professional Growth

5.1 Technical Skills Developed

- React
- FastAPI
- Full-stack integration
- Chart.js

- Database design
- API development
- MySQL
- Cloud deployment
- Data analytics
- SQLAlchemy
- API testing
- Cloud hosting platforms (Railway and Vercel)

Throughout this project, I gained hands-on experience developing a complete full-stack application from initial planning through development. I strengthened my skills in React frontend development, FastAPI backend development, relational database design, API development, data visualization, cloud deployment, and analytics implementation. I also gained experience integrating multiple technologies into a single system and troubleshooting issues that occurred during development and deployment.

5.2 Personal and Professional Growth

This project helped improve my time management, communication, and problem-solving skills. It also gave me more experience explaining technical ideas through presentations, meetings, and reports.

5.3 Connection to Coursework

This project connected concepts from several courses within both my Applied Computing major and Data Science & Analytics minor. Database concepts were applied through relational database design and SQL queries. Web development concepts were used when designing the frontend user experience and backend APIs. Data visualization concepts were applied through analytics dashboards and Chart.js visualizations. Analytics coursework influenced the development of recommendation generation and business insight features based on appointment data.

5.4 Career Impact

This project strengthened my interest in software development and data-focused roles by allowing me to combine full-stack development, database design, analytics, and cloud deployment within a single application. It also provided experience building a project from initial

planning through deployment and testing, which more closely reflects real-world software development.

An additional benefit of this project was the opportunity to identify a real-world problem through my experience working at a nail salon. Rather than creating a project for the sake of creating or completing the capstone course, I was able to observe an existing appointment management challenge and discuss potential solutions with the salon owner. Through stakeholder meetings and ongoing feedback from other meetings with the expert and instructor, I gained experience gathering business requirements, understanding user needs, and turning those requirements into a solution. This experience has helped me understand how the use of software can solve real business solutions and has strengthened my understanding of how important it is to have stakeholder and other continuous communication throughout the development process.

5.5 Biggest Takeaways

One of my biggest takeaways from this project was gaining a stronger understanding of how frontend, backend, database, analytics, deployment, and user experience components work together within a full-stack application. I also learned the importance of planning system architecture early and continuously testing features throughout development.

6. Key Insights and Future Directions

6.1 Most Important Takeaways

- Importance of planning architecture early
- Value of iterative development
- Importance of usability and workflow simplicity

6.2 Most Meaningful Part of the Project

One of the most meaningful parts of this project was building something connected to a real business and receiving feedback from an actual stakeholder. It was rewarding to develop a system that addressed real appointment management challenges while also incorporating analytics features that could support business decision-making. Seeing how my technical skills that I have been

working on throughout my college career could be applied to a real-world project that I experienced first hand made this project have more meaning than just being a school project.

6.3 Future Improvements

- Authentication/login system
- Staff scheduling
- SMS/email notifications
- Advanced analytics
- Customer history tracking
- Performance optimization of admin dashboard

Future improvements include combining multiple dashboard request into an aggregated endpoint to improve performance on deploying on the cloud.

6.4 Real-World Applications

A real salon could use this system to organize bookings more efficiently, reduce missed appointments, and better understand business trends through analytics and recommendations.

6.5 Advice for Future Students

I would recommend starting early, planning the project architecture carefully, and testing features incrementally instead of trying to build everything at once.

6.6 What I Would Do Differently

If I were starting this project again, I would spend more time planning the frontend structure, deployment strategy, and overall system architecture before implementation. I would also begin cloud deployment testing earlier in the development process to identify hosting and integration issues sooner.

7. References

- [React Documentation](#)
- [FastAPI Documentation](#)
- [Chart.js Documentation](#)
- [MySQL Documentation](#)
- [SQLAlchemy Documentation](#)
- [Railway Documentation](#)

- Vercel Documentation
- Pandas Documentation
- OpenAI ChatGPT (used for development assistance and generating realistic testing data)

8. Appendices

Appendix A - System Architecture

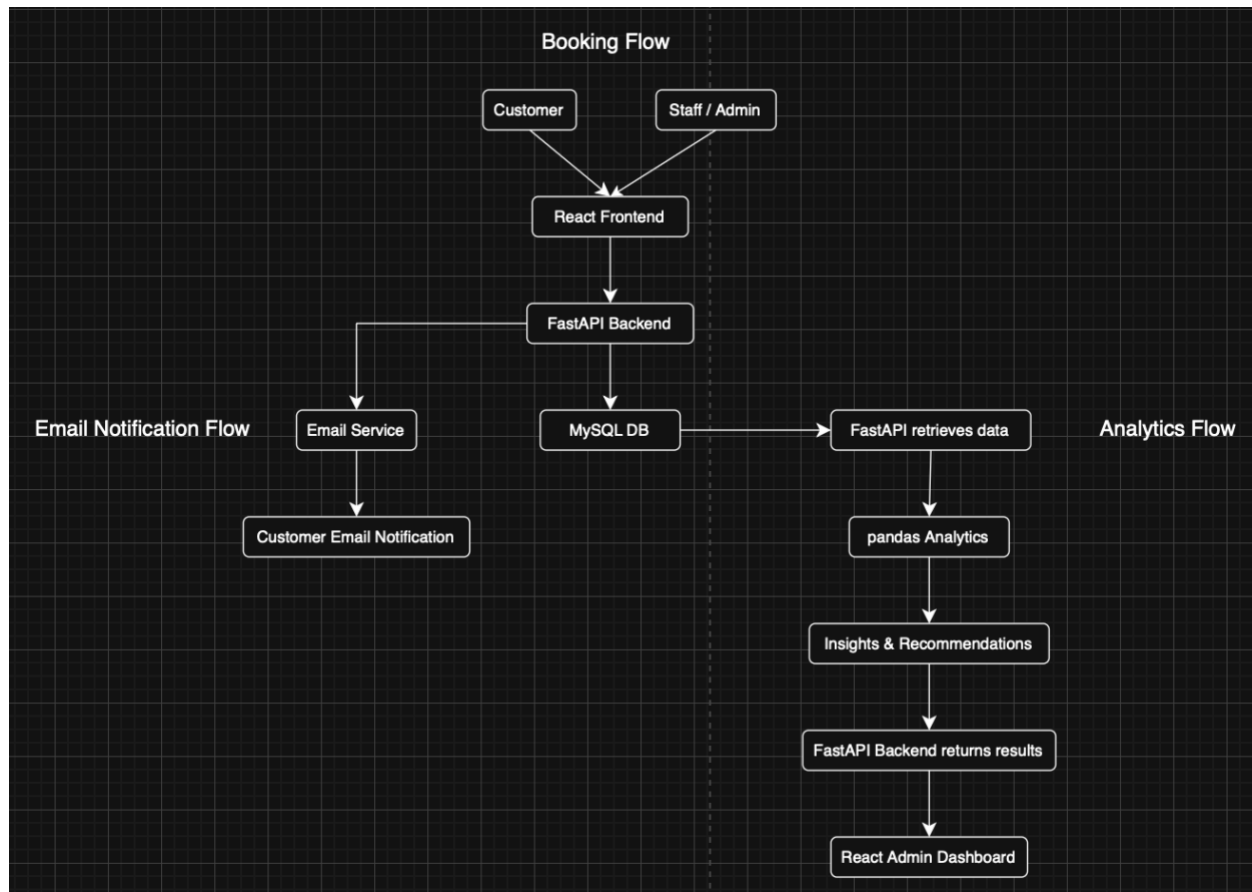


Figure A1. High-level system architecture showing the booking workflow, analytics workflow, and email notification workflow.

Appendix B - Database Schema

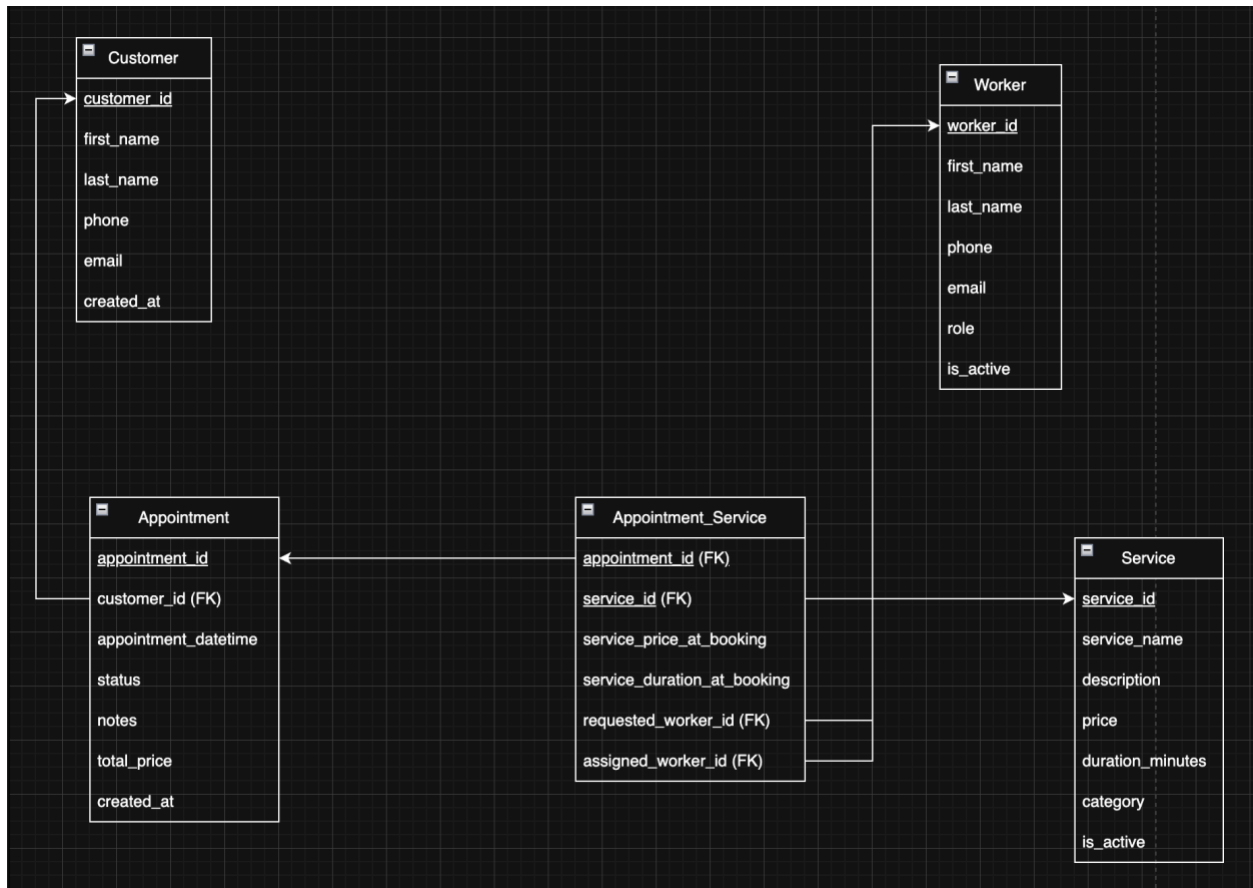


Figure B1. Final relational database schema showing the relationships between customers, appointments, services, workers, and appointment_services.

Appendix C - Customer Interface

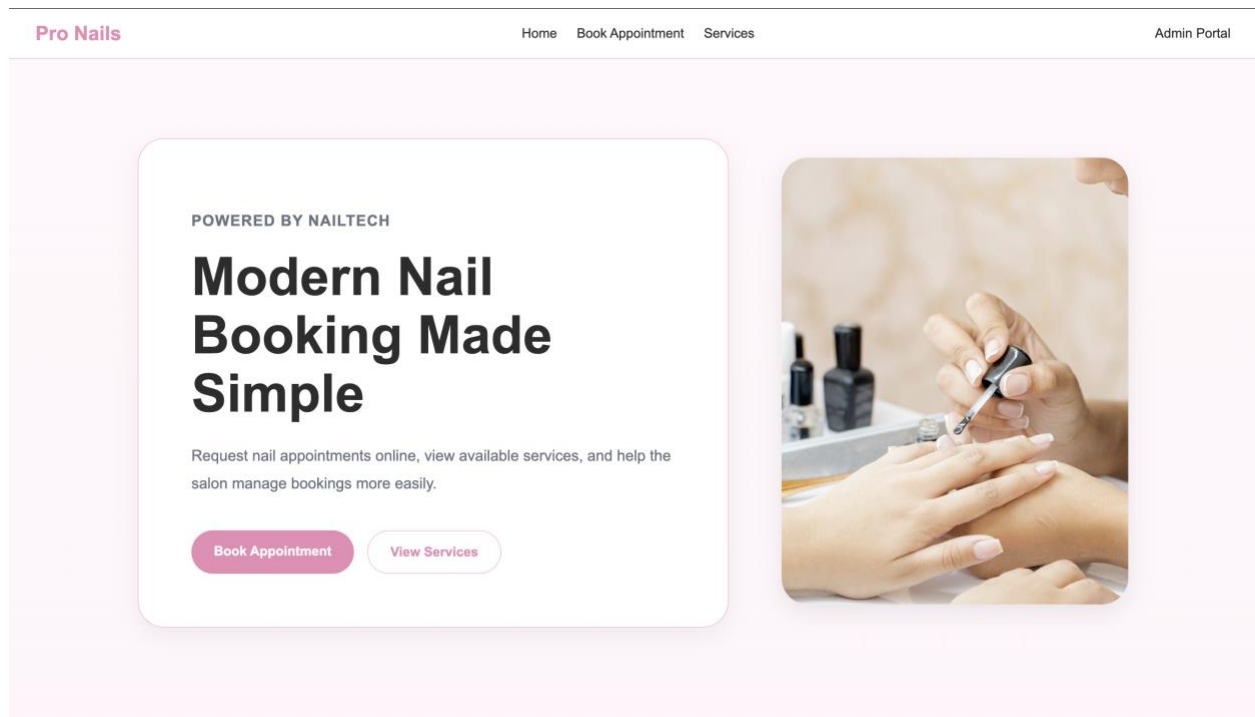


Figure C1. Homepage

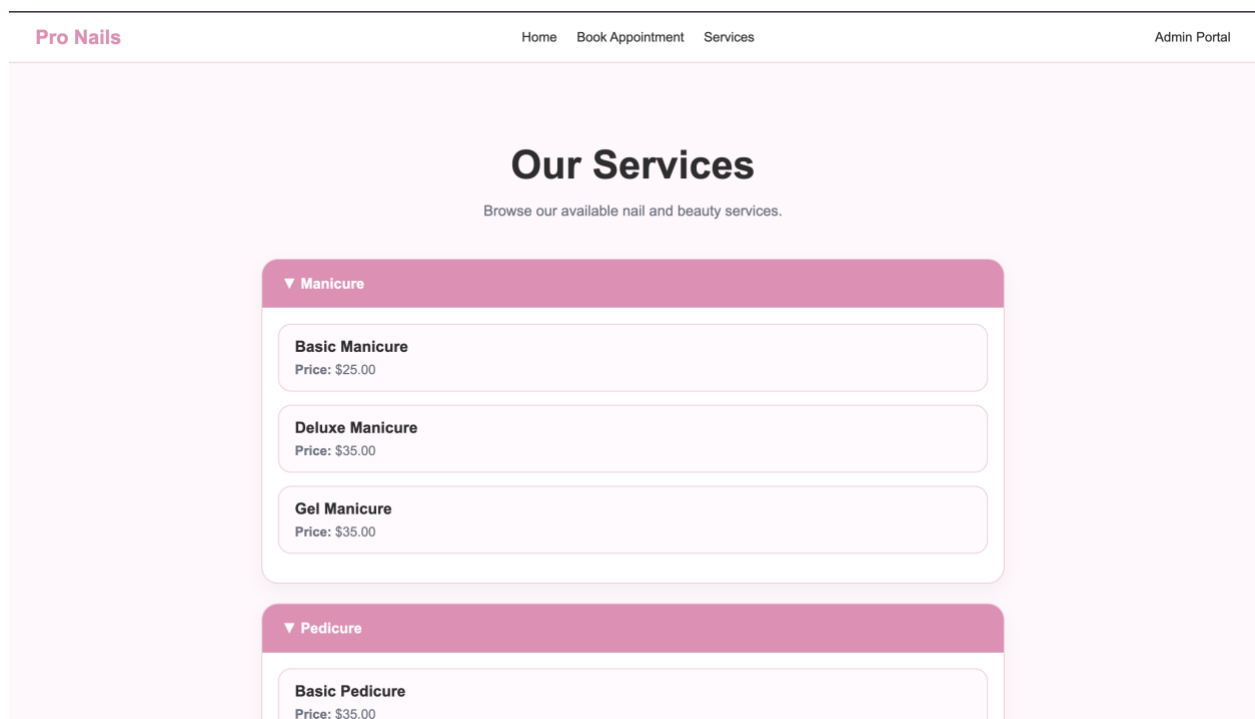


Figure C2. Services

Pro Nails
Home
Book Appointment
Services
Admin Portal

Book an Appointment

Enter your information, choose one or more services, and select your preferred time.

First Name

Last Name

Phone

Email

Services

Service 1

Requested Worker

Select a service

No preference

Estimated Total: \$0.00

Appointment Date

Appointment Time

mm/dd/yyyy

--:--

Notes

Add any notes here...

Book Appointment

Figure C3 – Booking Form

Appendix D – Admin Dashboard

Pro Nails
Home
Book Appointment
Services
Admin Portal

Admin Dashboard

Manage appointment requests and statuses.

Dashboard
Add Booking
Analytics

Refresh Appointments

Search by name, phone
All Statuses
mm/dd/yyyy
Clear Filters

ID	Customer	Phone	Email	Services / Requested Worker	Date & Time	Status	Notes	Actions
77	Megan Taylor	2069991016	megan@email.com	• Collagen Pedicure Requested: No preference	6/12/2026, 5:00:00 PM	Pending	Requested evening slot	Approve Decline Cancel Complete
75	Sophie Chen	2061111111	sophie@email.com	• Gel Manicure Requested: No preference	6/8/2026, 10:00:00 AM	Pending	Future repeat customer request	Approve Decline Cancel Complete
71	Kaitlyn Reed	2069991033	kaitlyn@email.com	• Gel Manicure Requested: Amy Nguyen	6/3/2026, 4:00:00 PM	Pending	Waiting for confirmation	Approve Decline Cancel Complete
70	Diana Vo	2069991032	diana@email.com	• Basic Pedicure Requested: No preference	6/2/2026, 2:00:00 PM	Pending	Requested afternoon time	Approve Decline Cancel Complete
47	Sophie Chen	2061111111	sophie@email.com	• Deluxe Manicure Requested: No preference	4/28/2026, 5:00:00 PM	Pending	After work	Approve Decline Cancel Complete

Figure D1. Admin Dashboard

Appendix E – Analytics Dashboard

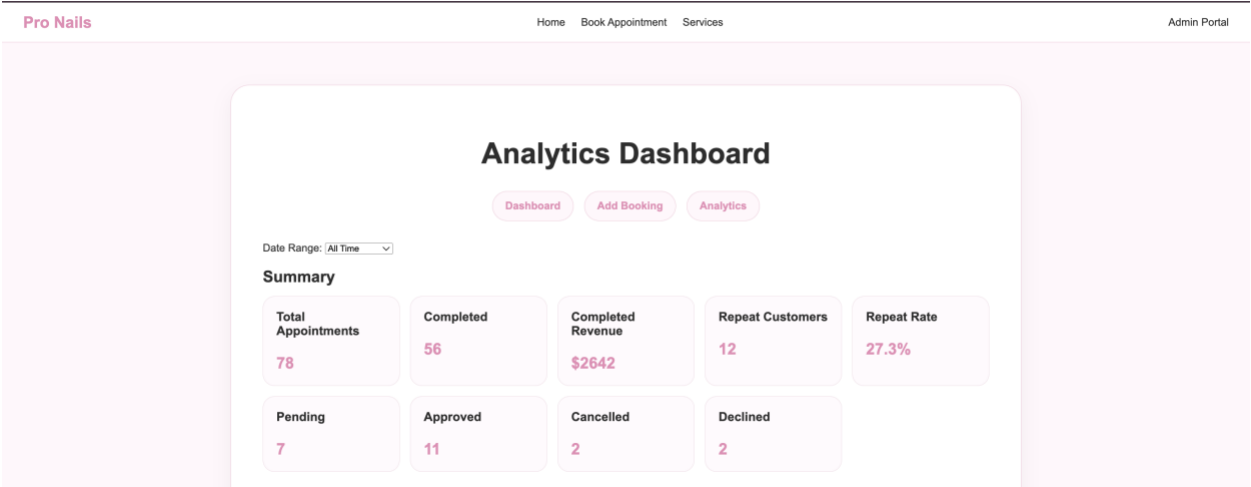
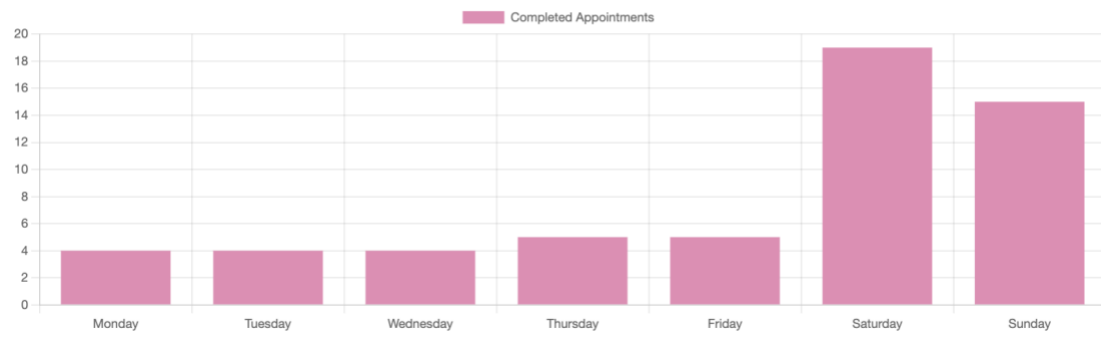
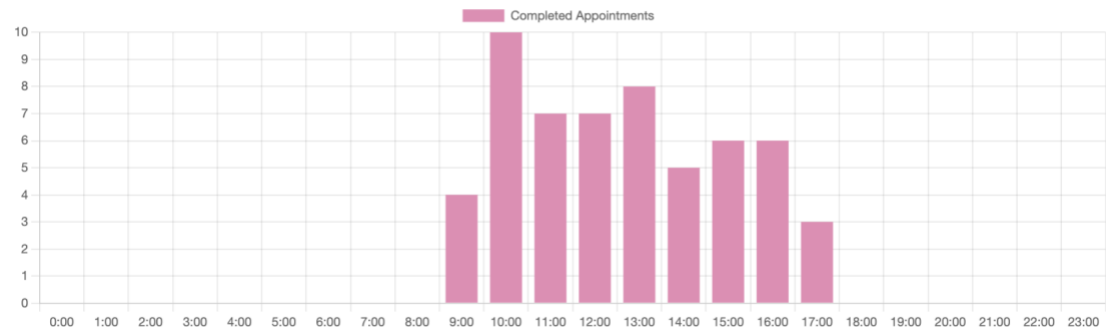


Figure E1 – Analytics Summary Cards

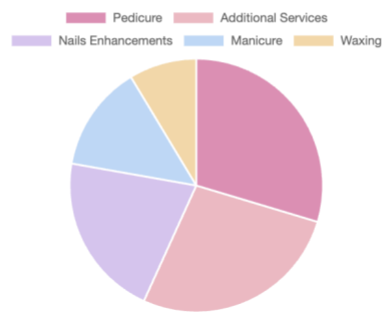
Busiest Days



Peak Hours



Service Categories



Top Services

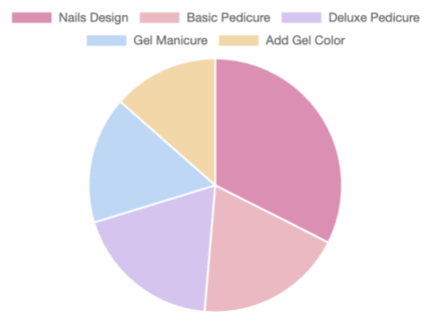
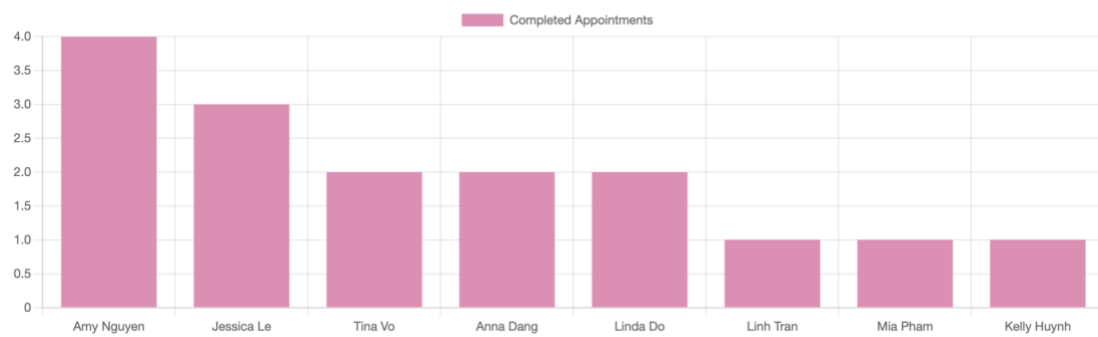


Figure E2 - Analytics Visualizations

Top Requested Workers



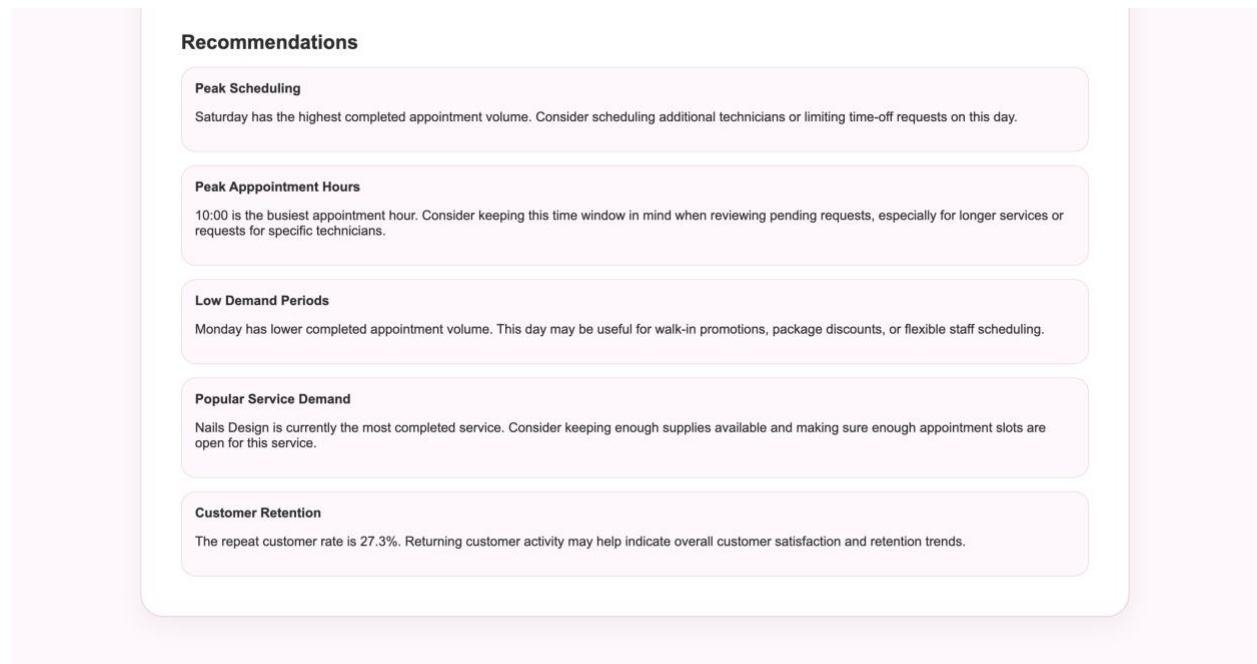


Figure E3 – Recommendation Engine

Appendix F - Email Notification Example

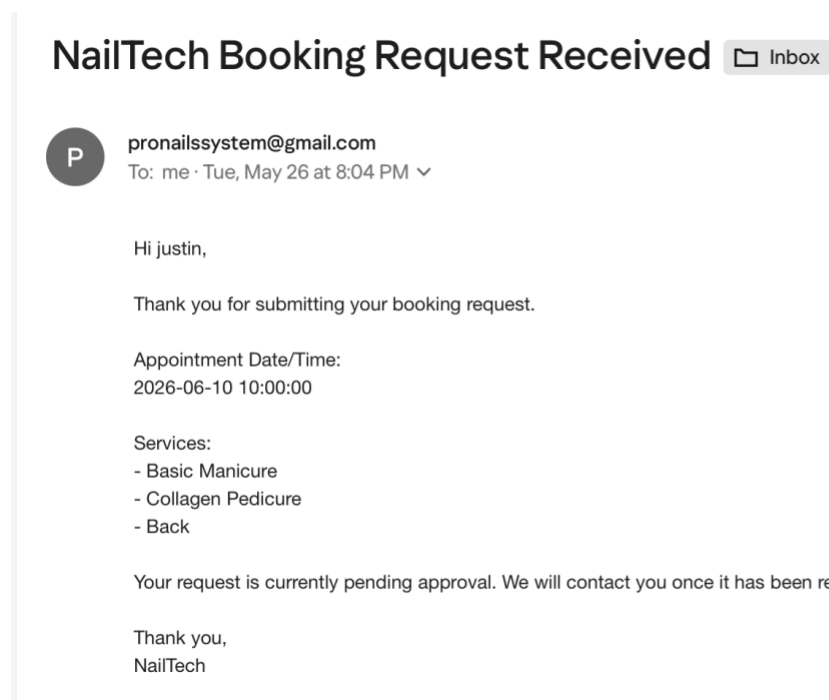


Figure F1. Appointment Status Email **Show approved or declined (current one is pending, but still works)**

Appendix - Sample API Responses

Figure G1 - POST /booking-request

Response

Figure G2 - GET /analytics/summary

Response

Appendix H – Sample Testing Data

appointment...	customer_name	appointment_dateti...	status	service_name	service_price_at_book...	service_duration_at_book...	requested_work...	assigned_worker	total_pri...
78	Naomi Young	2028-06-13 12:00:00	approved	Organic Volcano Gold Spa Pedicure	70.00	75	NULL	Linda Do	70.00
77	Megan Taylor	2028-06-12 17:00:00	pending	Collagen Pedicure	55.00	60	NULL	NULL	55.00
76	Ashley Kim	2028-06-09 15:30:00	approved	Jelly Pedicure	65.00	75	NULL	Tina Vo	65.00
75	Sophie Chen	2028-06-08 10:00:00	pending	Gel Manicure	35.00	45	NULL	NULL	35.00
74	Julie Park	2028-06-07 11:00:00	declined	Acrylic Full Set	35.00	75	Anna Dang	NULL	35.00
73	Monica Chen	2028-06-06 13:00:00	cancelled	Herbal Detox Pedicure	75.00	75	NULL	NULL	75.00
72	Sarah Lam	2028-06-06 10:30:00	approved	Pink & White Full Set	50.00	90	NULL	Anna Dang	50.00
71	Kaitlyn Reed	2028-06-03 16:00:00	pending	Gel Manicure	35.00	45	Amy Nguyen	NULL	35.00
70	Diana Vo	2028-06-02 14:00:00	pending	Basic Pedicure	35.00	45	NULL	NULL	35.00
69	Cindy Pham	2028-06-01 15:00:00	approved	Dipping Powder	45.00	60	NULL	NULL	45.00
68	Kayla Thompson	2028-05-17 11:00:00	completed	Deluxe Pedicure	45.00	60	NULL	Tina Vo	45.00
67	Ava Wilson	2028-05-16 12:30:00	completed	Nails Design	10.00	15	NULL	Jessica Le	55.00
67	Ava Wilson	2028-05-16 12:30:00	completed	Gel Builder Full Set	45.00	75	Jessica Le	Jessica Le	55.00
66	Grace Hoang	2028-05-16 10:00:00	completed	Acrylic Full Set	35.00	75	Jessica Le	Jessica Le	45.00
66	Grace Hoang	2028-05-16 10:00:00	completed	Nails Design	10.00	15	NULL	Jessica Le	45.00

Figure H1. Displays a joined view of appointment and appointment_services records used during testing to verify relationships, service assignments, and booking details.